

Google Cloud Bigtable

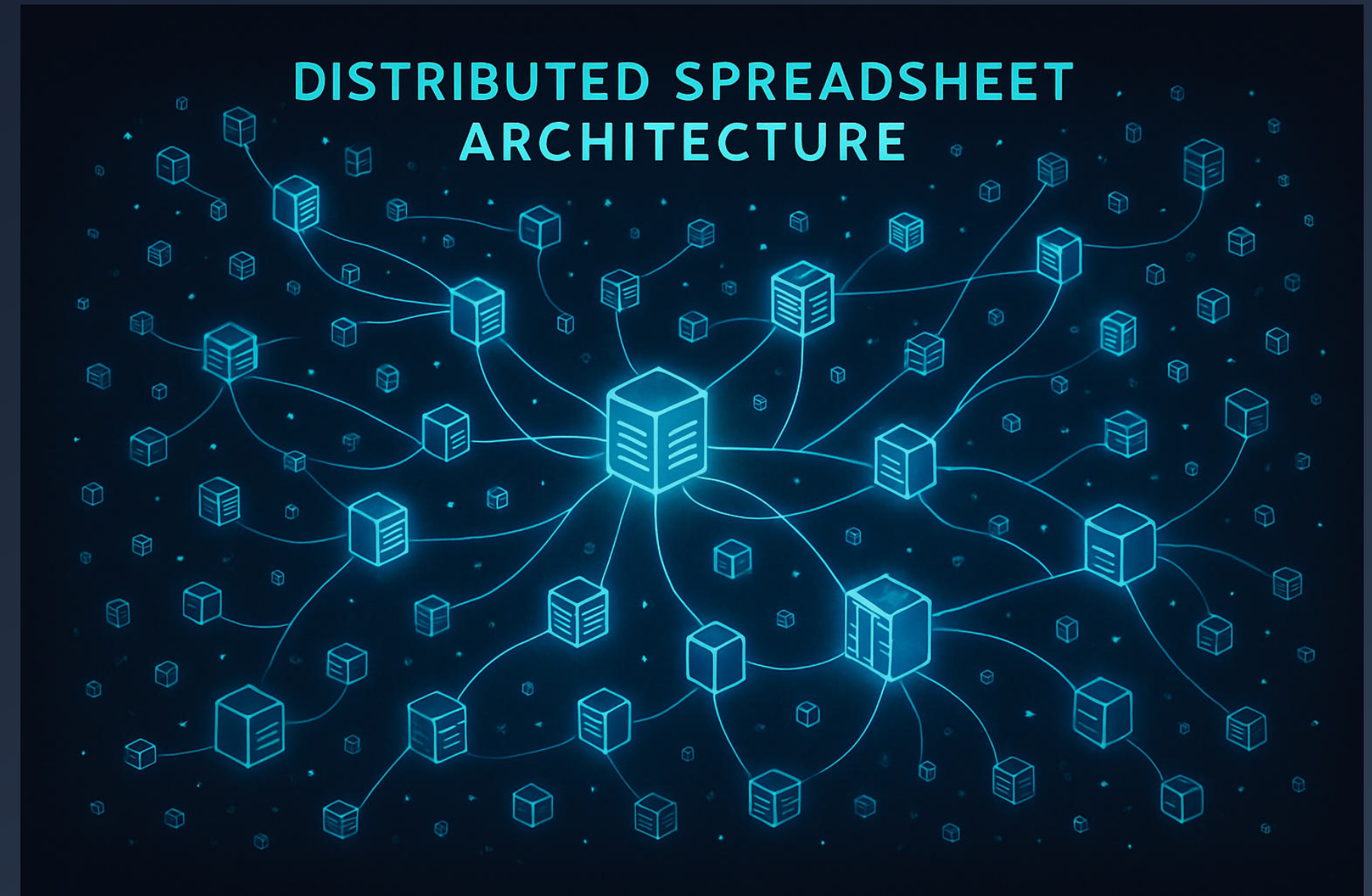


Base de Datos NoSQL Masivamente Escalable

¿Qué es Bigtable?

Base de Datos NoSQL de Google

-  Hoja de cálculo gigante distribuida
-  Cada fila identificada por Row Key única
-  Miles de millones de registros
-  Millones de lecturas/escrituras por segundo
-  Terabytes o Petabytes de datos



Productos que Usan Bigtable

Casos de Uso Reales de Google



Google Search

Índices de búsqueda web

Petabytes de datos indexados



YouTube

Métricas de visualización en tiempo real

Miles de millones de videos



Gmail

Indexación de correos electrónicos

Millones de usuarios activos

Potencia detrás de los servicios de Google

Modelo de Almacenamiento Columnar

Organización Lógica por Filas



Row Key

Clave única que identifica cada fila



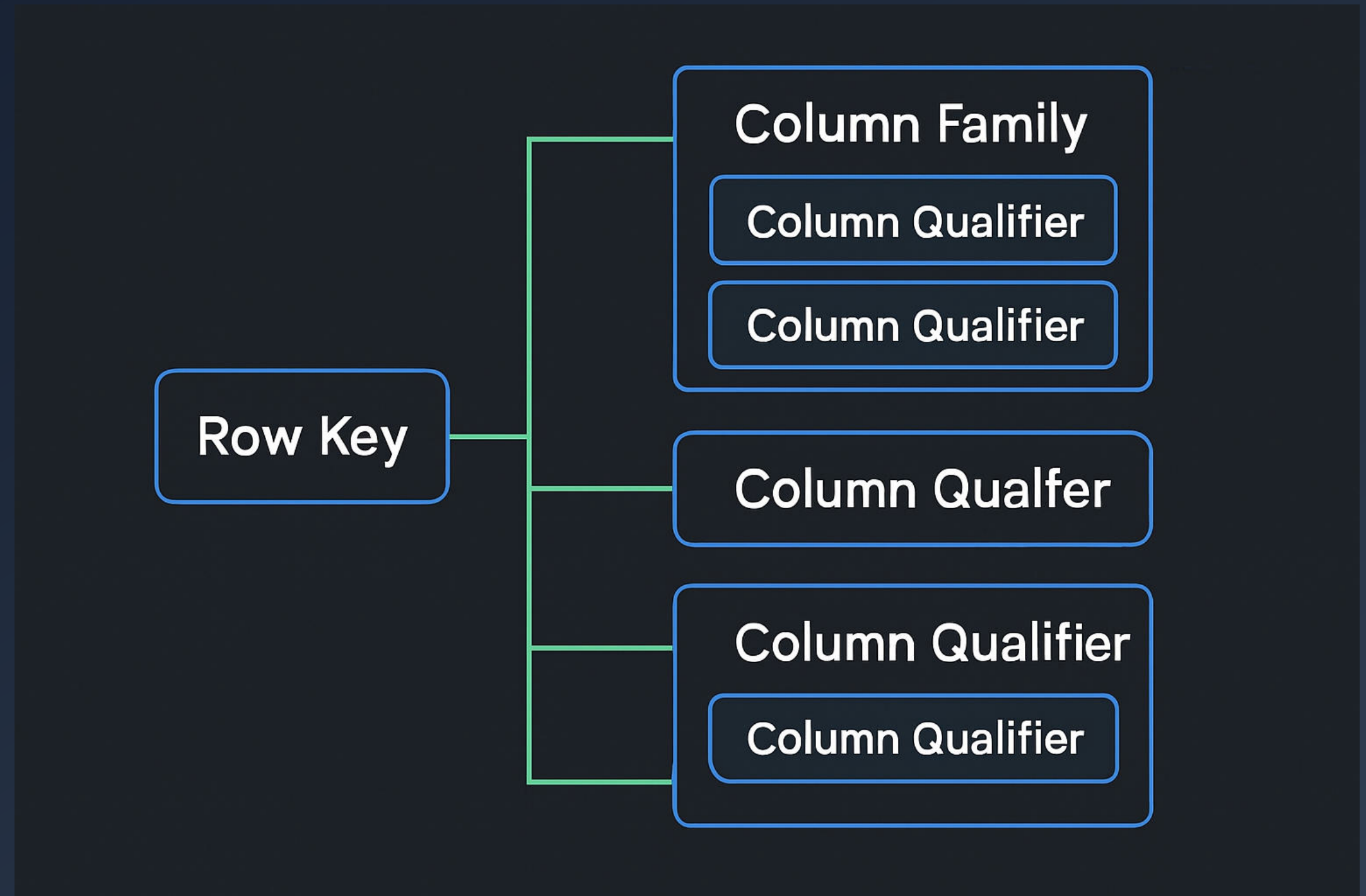
Column Families

Grupos de atributos relacionados



Column Qualifiers

Columnas específicas dentro de cada familia



Organización Física vs Lógica

Dos Perspectivas del Mismo Dato



Vista Lógica

Cómo tú ves los datos

Organizados por filas con Row Key

Lees y escribes por filas

	Column1	Column2	Column3
Row1	Data1	Data2	Data3
Row2	Data4	Data5	Data6
Row3	Data7	Data8	Data9
Row4	Data1	Data12	Data13



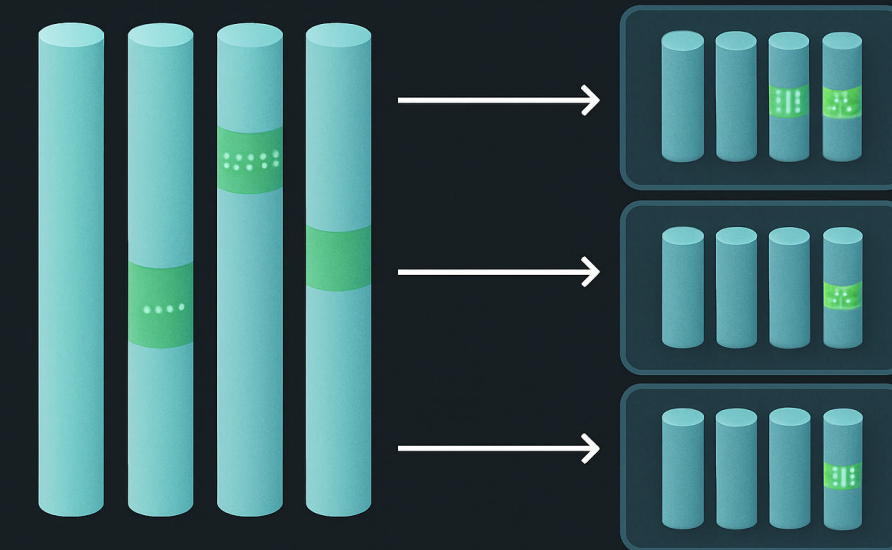
Almacenamiento Físico

Cómo se guardan los datos

Distribuidos por columnas

Compresión eficiente y lecturas rápidas

PHYSICAL COLUMNAR STORAGE



La mejor combinación de ambos mundos

Distribución en Nodos

Tablets y Escalabilidad Horizontal



División por Tablets

Filas divididas por rangos de Row Key



Asignación a Nodos

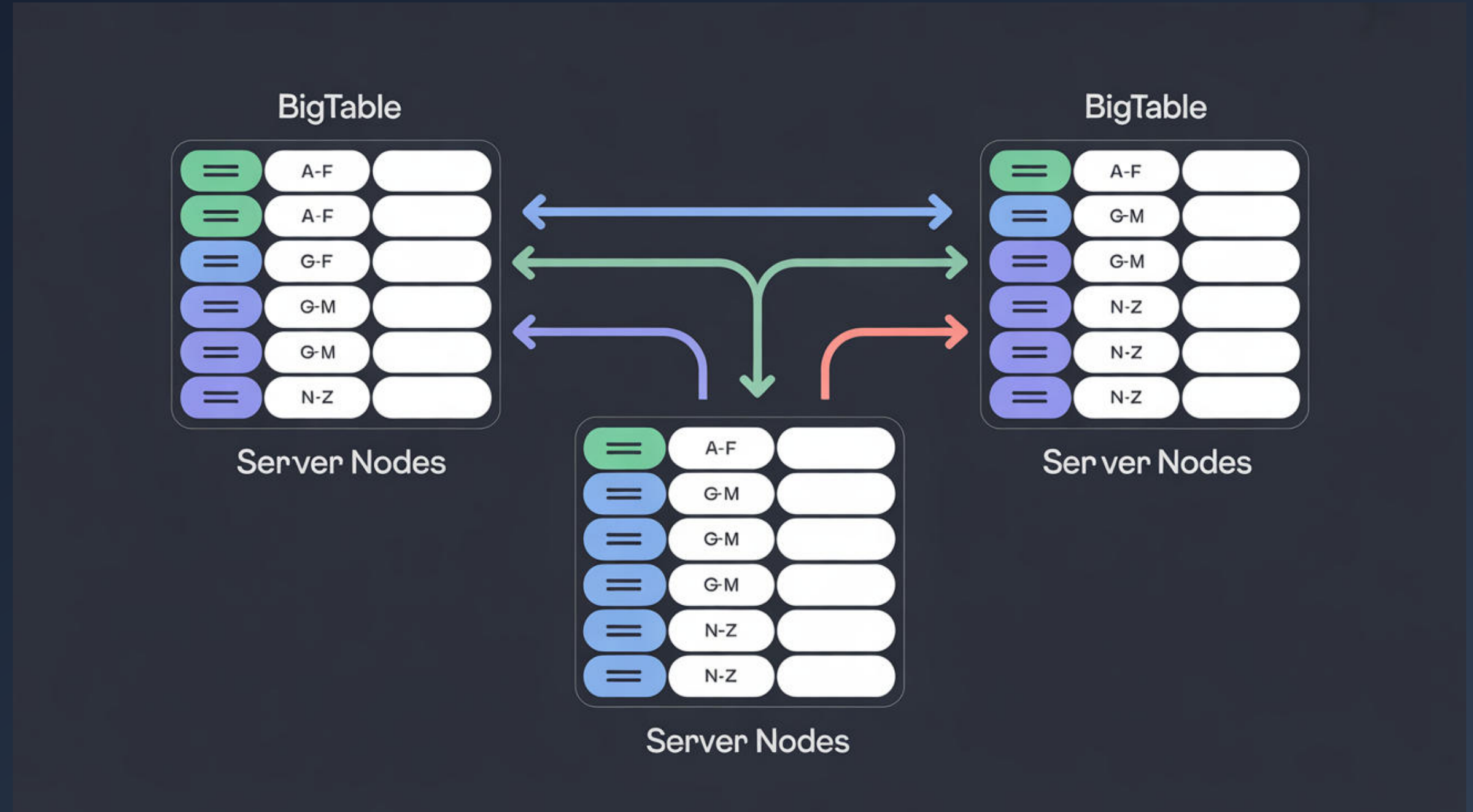
Cada tablet en un nodo diferente



Escalabilidad

Más nodos = más capacidad

El diseño de Row Key es crítico



Importancia de la Row Key

⚠ El diseño de la Row Key es LO MÁS IMPORTANTE ⚠



Rendimiento

Afecta la velocidad de lecturas y escrituras



Costo

Determina el uso eficiente de recursos



Escalabilidad

Define cómo crece tu sistema

De esto depende casi todo

Principios de Diseño de Row Key

Guía para Diseño Efectivo

1



Evitar Hotspots

No usar valores crecientes al inicio

✗ timestamp al principio

2



Soportar Rangos

Permitir filtrado eficiente

✓ agrupar por entidad

3



Uniformidad

Distribuir claves uniformemente

✓ usar salting o hashing

4



Pensar en Consultas

Diseñar para accesos comunes

✓ optimizar para tu uso

Ejemplos IoT: Row Key

Sensores IoT: Mal Diseño vs Buen Diseño



MAL DISEÑO

2025-10-07T17:00#sensor123

Problema: Hotspotting

Todas las claves con la misma fecha van al mismo nodo

Las escrituras se concentran en un solo punto



Cuello de botella



Bajo rendimiento



Nodo sobrecalentado



BUEN DISEÑO

sensor123#2025-10-07T17:00

Ventaja: Distribución Uniforme

Los datos se agrupan por sensor y permiten consultas por rango temporal

Las escrituras se distribuyen equilibradamente



Alto rendimiento



Escalable



Optimizado

Extra: Con Salting

a#sensor123#2025-10-07T17:00

Prefijo hash distribuye aún mejor

El orden importa: entidad primero, tiempo después

⚠️ ¿Qué es el Hotspotting? ⚠️

Problema de Distribución Desbalanceada

📄 Definición

Muchas operaciones se concentran en una sola parte del clúster

🔑 Causa Principal

Row Keys secuenciales (timestamps, IDs crecientes)

🚨 Ejemplo Problemático

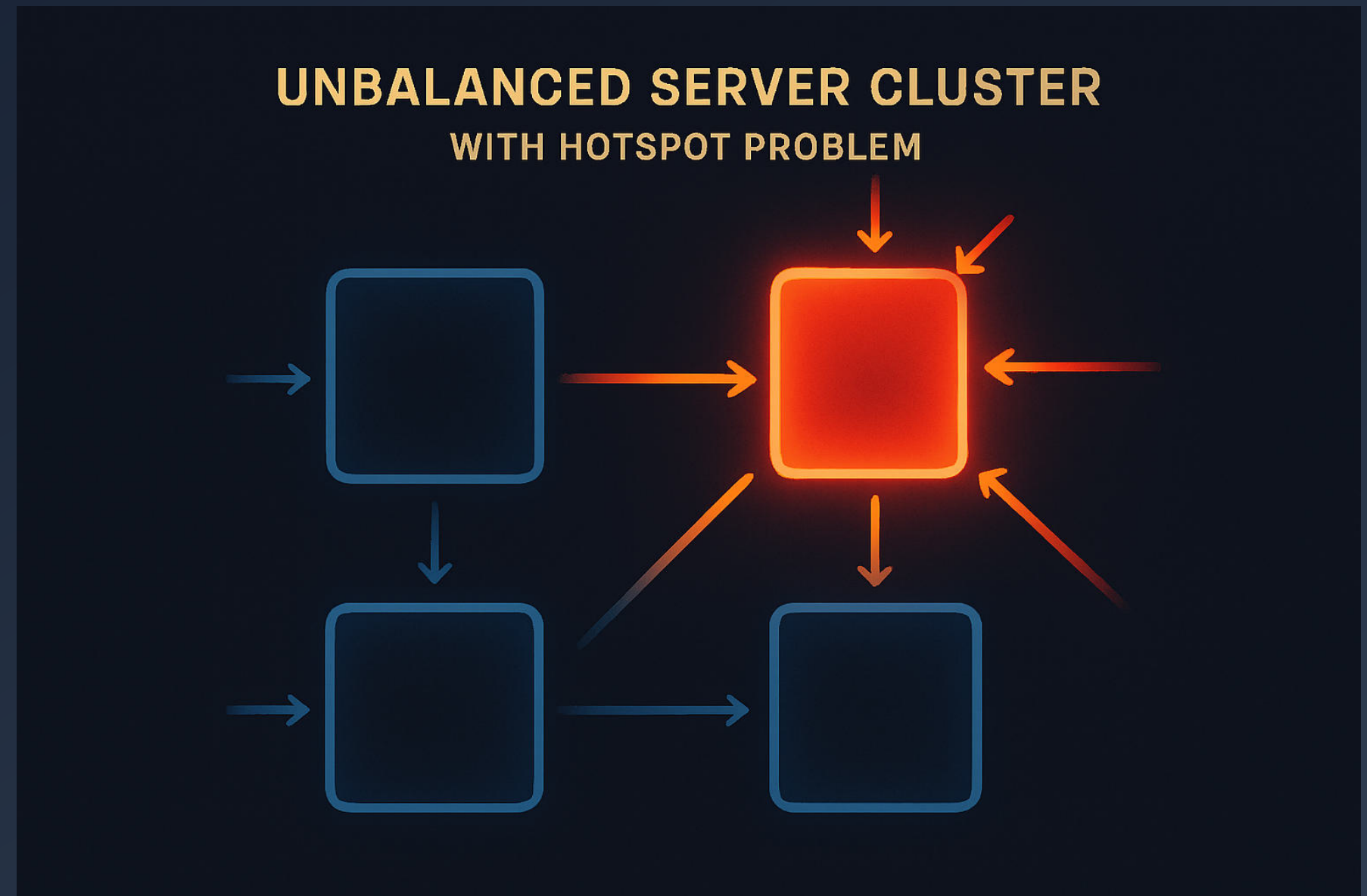
2025-10-07T17:00

2025-10-07T17:01

2025-10-07T17:02

🔥 Resultado

Un nodo recibe todas las escrituras mientras otros están vacíos



Un nodo sobrecargado arruina todo el rendimiento

Cómo Evitar Hotspotting

✓ Técnicas para Distribución Balanceada

1



Invertir el Orden

Colocar la entidad primero, no el timestamp

2025-10-07#sensor123

ANTES



sensor123#2025-10-07

DESPUÉS



Las escrituras se distribuyen por sensor

2



Salting (Prefijo Hash)

Añadir prefijo aleatorio basado en hash

sensor123#2025-10-07

ANTES



a#sensor123#2025-10-07

DESPUÉS



Distribución uniforme garantizada

3



Field Promotion

Promover campos de alta cardinalidad al inicio

timestamp#user_id#event

ANTES



user_id#timestamp#event

DESPUÉS



Claves bien distribuidas naturalmente



Distribución uniforme = máximo rendimiento

Características Clave

Por qué elegir Bigtable



Modelo de Datos

Wide-Column NoSQL

Cada fila con Row Key única



Escalabilidad

Horizontal ilimitada

Billones de registros



Baja Latencia

Milisegundos

Millones ops/segundo



Persistencia

Datos en SSD

Replicación entre zonas



Integración

Ecosistema GCP

Dataflow, BigQuery, AI



API Compatible

HBase open-source

Clientes existentes

Optimizada para escala masiva y baja latencia

Bigtable vs Firestore

Comparación de Bases de Datos NoSQL

Característica	Bigtable	Firestore
Tipo	Wide-Column (Cassandra/HBase)	Documental
Casos de Uso	Analítica, IoT, logs, series de tiempo	Apps web/móvil, sincronización
Escala	Petabytes, millones de ops/seg	GBs, millones de docs (limitado)
Consultas	Por Row Key o rangos de keys	Múltiples campos e índices
Transacciones	Limitadas a una sola fila	Sí, con consistencia fuerte
Costo	Alto pero muy escalable	Bajo para apps medianas
Interfaz	SDK/API, sin UI nativa	UI completa en Firebase Console
Integración	Dataflow, BigQuery, Vertex AI	Firebase, Cloud Functions

Dos herramientas, dos propósitos diferentes

Casos de Uso Prácticos

¿Cuándo usar cada una?



Bigtable

Para Escala Masiva



Métricas IoT

Millones de lecturas de sensores por segundo



Series Temporales

Datos de monitoreo y analítica en tiempo real



Logs

Análisis de logs a gran escala



Analítica

Procesamiento de petabytes de datos

Ejemplo Real

sensor123#2025-10-07T17:00

temperature: 24.5, humidity: 82

Análisis de métricas a gran escala



Firestore

Para Apps Web/Móvil



Apps Móviles

Datos sincronizados en tiempo real



Chat

Mensajería con consultas complejas



E-commerce

Pedidos con seguridad por usuario



Social

Perfiles y contenido estructurado

Ejemplo Real

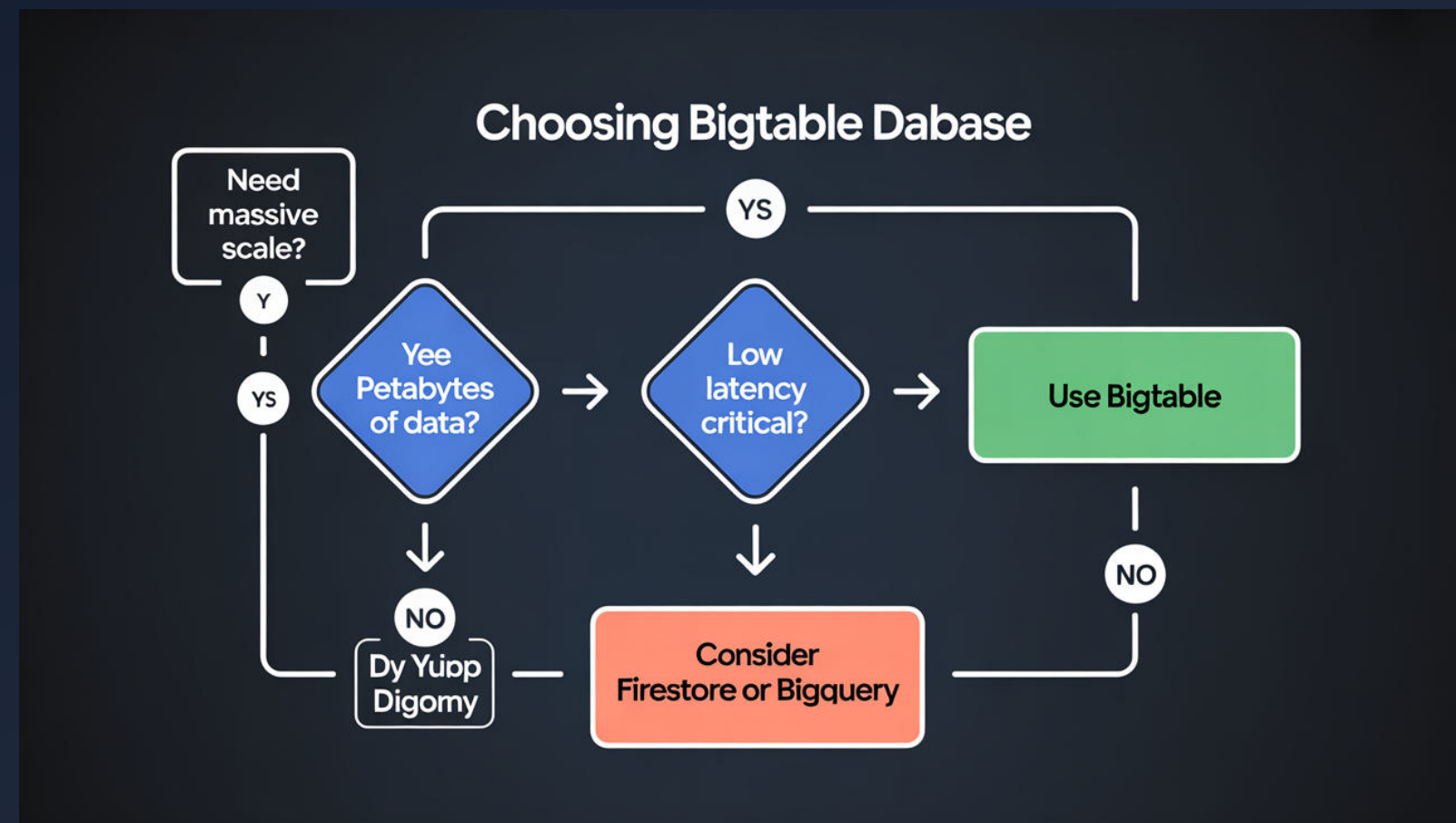
```
{"user_id": "u123", "message": "Hola!"}
```

Consultas complejas y sincronización

Elige según tu caso de uso y escala

¿Cuándo Elegir Bigtable?

Guía de Decisión Rápida



Puntos Clave para Recordar



Bigtable es para escala masiva y baja latencia



El diseño de Row Key es crítico para el rendimiento



Evita hotspotting con buenas prácticas



Escalabilidad horizontal ilimitada



Bigtable: La base de datos para aplicaciones de escala planetaria

